Troubleshooting Guide

Neighbourhood Livability Index

Implementation lessons and suburb-boundary accuracy guidance | 19 October 2026 | Current app commit 4229b1d

Scope and first response

This guide records implementation problems seen while building the two-suburb Parks comparison and gives a repeatable way to diagnose them. The current app uses Streamlit, Supabase PostgREST, the ABS suburb catalogue and OSM Overpass. It counts leisure=park, leisure=garden, leisure=nature_reserve and leisure=playground within a CSV latitude/longitude rectangle. Google Places and its 60-result cap are not active in this release.

Triage order

Order	Check	Evidence to capture
1	Does the app start and render the comparison controls?	Terminal error, app URL and selected Git commit.
2	Which layer supplied the result: Streamlit cache, Supabase or live Overpass?	Displayed source, refresh action, and matching Supabase updated_at row.
3	Did the API return data and did the database save it?	HTTP status and sanitized response body; never capture keys or tokens.
4	Are the selected suburb bounds and area correct?	SAL code, state, CSV min/max coordinates, area_km2 and boundary version.
5	Is the KPI result accurate for the business definition?	OSM tag filter, unique feature identities, count, density and score calculation.

Useful distinctions

- A SQL result of 0 rows may mean a SELECT found no matching rows, or a DDL/DML command reported 0 rows changed. Check the statement and then run a SELECT against the exact table.
- A value of 5 from COUNT(*) is a number of records, not necessarily five parks in every suburb. Read place_count by suburb and source to validate KPI values.
- An OpenStreetMap source label identifies the original data provider; the current UI does not expose a separate cache-layer label for every in-process cache hit.

App startup and Overpass problems

Symptom seen	Likely area	Troubleshooting and recovery
Streamlit fails with SyntaxError such as expected except or finally	Python indentation or an incomplete try/except after an edit.	Run python -m py_compile app.py. Correct the indicated block, save, run the compile check again, then restart Streamlit.
Browser cannot open localhost:8501	App is stopped, still starting, listening on another port, or crashed at startup.	Read the terminal that launched Streamlit. Run python -m streamlit run app.py. If 8501 is occupied, try --server.port 8502 and open that URL.
406 Not Acceptable from overpass-api.de	Overpass endpoint rejected the request or was temporarily unavailable.	Check the request method, query and Accept: application/json header. Retry once; the app tries configured endpoints. Do not interpret 406 as zero places.
502 Bad Gateway from Overpass	Upstream Overpass server or proxy failed temporarily.	Retry later or use saved Supabase counts. Use Refresh live OSM counts only when a fresh provider result is needed. Keep the failure visible; never relabel saved data as live.
ProxyError to 127.0.0.1:9 / connection refused	HTTP_PROXY/HTTPS_PROXY/ALL_PROXY may point to an unavailable local proxy, or network policy blocks outbound traffic.	Inspect proxy variable names and the managed network configuration. Correct or remove only confirmed stale settings; do not bypass a required corporate proxy. Retest HTTPS to Supabase and Overpass.

Local startup checks

```
python -m py_compile app.py
python -m streamlit run app.py
python -m streamlit run app.py --server.port 8502
```

Overpass does not use Google Places' fixed 60-result pagination cap. It can return more than 60 matching elements, but public endpoints have resource limits, queue delays and timeouts. Large rectangles may fail even when the tag query is valid.

Supabase and CSV import problems

Symptom seen	Check first	Fix or verification
400 Bad Request while saving live counts	PostgREST response detail, updated_at column, data types, upsert conflict key and RLS source allowlist.	The current app sends suburb, category, place_count, source and updated_at and upserts on (suburb, category). Apply the timestamp migration if needed; ensure policies allow the current source. Refresh PostgREST schema metadata if columns changed.
SQL query returned 0 rows	Whether the query targets suburbs_import, suburbs or amenity_counts; filters and transaction result.	After CSV import, check staging row count first, run the bounds sync, then check active public.suburbs. For counts, query amenity_counts separately.
suburbs_import not visible	Correct Supabase project, public schema, table creation SQL and Table Editor refresh.	Run supabase_suburb_bounds.sql in the intended project, refresh the schema view, then verify with SELECT count(*) FROM public.suburbs_import.
CSV import succeeded but app search lacks suburbs	Import was staged but sync SQL was not run, or rows are inactive/missing boundaries.	Run supabase_suburb_bounds_sync.sql, then check is_active, area_km2 and min/max coordinates in public.suburbs. Confirm app SUPABASE_URL points to this project.
Query shows 5 for each suburb	COUNT(*) may count returned rows or a grouped dataset; sample test rows may also have equal values.	Select suburb, category, place_count, source, updated_at directly. Filter to osm_overpass_bbox_green_recreation and compare the actual place_count values.

Read-only SQL checks

```
select count(*) as staged_rows from public.suburbs_import;
```

```
select count(*) as active_suburbs from public.suburbs where is_active = true;
```

```
select suburb, category, place_count, source, updated_at
from public.amenity_counts
where suburb in ('Chatswood, NSW', 'Newtown, NSW')
order by suburb, updated_at desc;
```

Use the exact suburb labels stored in the table. Do not paste API keys or password values into SQL output, screenshots or support messages.

Cache and source-of-data questions

Current lookup sequence

Layer	Behavior	Lifetime / limitation
Streamlit result cache	Returns a cached result for the same comparison call before the function executes again.	24 hours in a running app process; not shared across independent instances and cleared by process restart/redeploy.
Supabase counts	Read after a process cache miss; successful live results are upserted for later sessions/processes.	updated_at is stored, but current app reads do not enforce a max age.
Live Overpass	Queried when selected current-source counts are missing or user requests Refresh live OSM counts.	Public endpoint availability and latency vary; successful counts are synced to Supabase.
Legacy Supabase rows	Older circular park-only rows may be used if live/current-source lookup fails.	They have a different method and should remain visibly labeled as legacy.

How to confirm the source

1. Read the result caption: Supabase means the function returned a saved count; OpenStreetMap means the underlying count came from a live query.

2. Check source and updated_at for each selected suburb in public.amenity_counts. A recent timestamp supports a recent sync but does not by itself prove the latest screen reran the query.

3. Use Refresh live OSM counts, then confirm that counts and updated_at changed and that the app reports a live OSM result. If the request fails, expect a warning and a saved-data fallback where available.

4. For production diagnostics, add an explicit retrieval_layer field or event log with values such as process_cache, supabase_cache, live_overpass and legacy_fallback.

Why identical counts can appear

- An aggregate SQL COUNT can be mistaken for place_count. Query both columns and group by suburb only when that is the actual question.
- Illustrative rows, legacy circular rows and current four-tag rectangle rows use different definitions. Filter by source before comparing values.
- A process cache can preserve the first result's provider label. The current caption does not fully distinguish live-origin data served from the local cache; use the explicit refresh and database timestamp checks above.

Accuracy limitation of rectangular bounds

Why it affects the KPI

The current suburb CSV supplies min_lat, min_lon, max_lat and max_lon. These coordinates form a rectangle around the suburb, not its actual SAL shape. For an irregular, coastal or narrow suburb, the rectangle can cover neighboring areas, water or places outside the SAL. The numerator can therefore be too high while the denominator remains the official SAL area_km2, biasing features per square kilometre and the pair-relative score.

The amount of bias differs by suburb. If the rectangle adds many out-of-suburb features for one comparison suburb but few for the other, their rank or score gap can be distorted. More precise arithmetic on the same rectangle cannot remove this geographic error.

Recommended accuracy improvement

Replace the rectangle-only inclusion rule with official ABS ASGS SAL polygon geometry for the same boundary vintage as the suburb catalogue. Keep the rectangle as a fast candidate-search envelope, then perform an exact spatial inclusion test against the selected SAL polygon before counting.

Current method	Recommended method
Query features in a min/max rectangle and count the returned representative centers that fall in that rectangle.	Query candidate features in the rectangle, obtain usable feature geometry or a canonical point, then spatially filter against the exact SAL polygon.
Use suburb text as the primary count key.	Use the ABS SAL code as the stable join key for suburb, boundary and count records.
Use CSV area_km2 even when the numerator includes outside places.	Use one documented, versioned SAL area definition. Keep the boundary and denominator from the same approved source/version.

Choose how a place belongs to a suburb

- For point-like features, use the OSM node coordinate. For area features such as mapped parks, retrieve actual way/relation geometry and derive a representative point that lies on the feature, for example ST_PointOnSurface in PostGIS.
- Count a feature for the SAL that covers its selected representative point. Define an explicit rule for points exactly on a shared boundary so the same feature is not inconsistently assigned to neighboring suburbs.
- If instead the product wants every park that overlaps a suburb, use geometry intersection, but disclose that a park crossing a SAL boundary can be counted in multiple suburbs. Do not mix the overlap and representative-point rules.

Polygon implementation plan

1. Obtain the official SAL polygon dataset for the same ASGS release as the suburb catalogue. Preserve SAL_CODE, source release and geometry metadata during import.

2. Add a PostGIS geometry column using SRID 4326 and MultiPolygon-compatible geometry, keyed to the SAL code. Validate geometry validity and add a GiST spatial index.

3. Use each SAL polygon's bounding envelope only to reduce the Overpass candidate query. Retrieve enough OSM geometry for the selected representative-point rule; Overpass out center alone may be an approximate center, not a precise point-on-surface of the feature.

4. Deduplicate candidates by OSM element type and ID. Convert nodes and ways/relations into one canonical geometry representation, retain tags and source timestamp, and spatially match to the selected SAL polygons.

5. Count only the approved leisure tags after exact inclusion. Record SAL code, boundary release, OSM source, tag definition, count method, fetched_at and count version so saved values remain interpretable.

6. Calculate density using an agreed area field for that same SAL release. Retain the existing score formula only if product owners approve its pair-relative meaning.

7. Backfill selected suburbs, compare old rectangle and new polygon results, and publish the new method as a versioned source. Do not silently overwrite historical values from a different method.

Spatial filter pattern

A typical PostGIS design stores SAL geometry in public.suburbs and candidate feature geometry in a staging or OSM feature table. The exact predicate can use ST_Covers(sal.geom, feature.representative_point) to include boundary points, provided shared-boundary assignment is handled. If the business definition is overlap rather than representative-point placement, use ST_Intersects and disclose possible cross-suburb double counts.

```
-- Illustrative query shape; adapt table and geometry column names
select s.sal_code, count(distinct (f.osm_type, f.osm_id)) as feature_count
from public.suburbs s
join staging.osm_features f
  on ST_Covers(s.geom, f.representative_point)
where s.sal_code = :selected_sal_code
  and f.leisure_tag in ('park', 'garden', 'nature_reserve', 'playground')
group by s.sal_code;
```

The query is a design example, not a migration ready to run. Confirm geometry validity, SRID, table names, supported Postgres/PostGIS version and duplicate handling before implementation.

Measure the accuracy improvement

Compare both methods fairly

Run the rectangle and polygon methods against the same OSM snapshot, same four tags, same deduplication rule and the same SAL area denominator. Otherwise a difference may come from data freshness or methodology rather than the boundary shape.

Measure	Calculation / use
Rectangle count	Unique matching OSM elements returned under current CSV rectangle rule.
Polygon count	Unique matching elements assigned by the approved SAL polygon rule.
Outside-rectangle share removed	100 x (rectangle count - polygon count) / rectangle count when rectangle count > 0.
Pair rank change	Whether the higher-density suburb changes after exact filtering.
Manual boundary review	Map a representative sample of candidate features near SAL edges and inspect false inclusions/exclusions.

Test a representative set

- Include coastal and irregular SALs, small and large suburbs, dense inner-city areas, islands and suburbs with concave boundaries.
- Sample features inside the polygon, in the rectangle but outside the polygon, close to shared SAL borders, and mapped as nodes, ways and relations.
- Use GIS software or an independent spatial query to inspect geometry overlays. Preserve the sample list and expected inclusion decisions as regression fixtures.
- Verify the `outside-rectangle share removed` is not negative. If polygon count exceeds rectangle count, check candidate envelopes, coordinate order, boundary vintage, query completeness and duplicate rules.

Remaining accuracy limits

Exact SAL geometry removes rectangle spillover; it does not make OSM complete or authoritative for every park. OSM mapping coverage, tag consistency, feature updates, overlapping mapped park/playground objects and feature geometry quality remain sources of uncertainty. The result should be described as a count of mapped OSM features under a stated rule, not as the total number of usable parks or a walking-access measure.

Prevention and quick reference

Release checklist

[ ] Run python -m py_compile app.py after code edits before starting Streamlit.

[ ] Keep .env, Streamlit secrets, Supabase keys and proxy credentials out of Git and screenshots.

[ ] Verify the app points to the intended Supabase project before running imports or SQL synchronization scripts.

[ ] Use read-only SQL to confirm staging, active suburb and amenity count rows before concluding an import or sync failed.

[ ] Label provider, retrieval layer, timestamp, tag set, boundary method and fallback status separately in production observability.

[ ] Do not treat Overpass 406/502, missing Supabase rows or a COUNT(*) aggregate as a valid zero feature count.

[ ] Keep current rectangle results and polygon results under distinct source/version values during migration and comparison.

[ ] Document stale cache policy, Overpass error handling, response-time target and accepted accuracy before production launch.

Current implementation references

- Repository: https://github.com/prasad830105/DEMOGRAFY_INT, main branch commit 4229b1d.
- App: app.py. Suburb and cache setup: README.md. KPI definition and current rectangle caveat: KPI-card.md.
- Supabase setup/migrations: supabase_schema.sql, supabase_suburbs.sql, supabase_suburb_bounds.sql, supabase_suburb_bounds_sync.sql, supabase_amenity_counts_updated_at.sql and the OSM RLS policy scripts.

Production recommendation: approve the OSM source and KPI definition, implement exact SAL polygon filtering with versioned data, and validate the overcount and rank-change measures before replacing the current rectangle estimate.